

Week-5

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Solution:

MapReduce has been used for a long time in big data processing, but it has several limitations that make Spark a better choice today.

1. Heavy dependence on disk

MapReduce reads data from the disk and writes the results back to disk at every stage of processing. This constant back-and-forth slows things down a lot, especially with large datasets.

2. Not suitable for real-time processing

MapReduce is designed for batch jobs, meaning it processes data in bulk after collecting it, not as it arrives. So it isn't the right tool when an application needs instant or quick results.

3. Poor performance for iterative calculations

Algorithms that need to go over the same data multiple times, like machine learning algorithms, perform badly under MapReduce, since it has to fetch the data from disk again and again for every pass.

4. Difficult to write complex programs

MapReduce only follows a simple two-step process — Map and then Reduce. Anything more complicated usually needs several MapReduce jobs chained together, which makes the code messy and harder to manage.

5. Higher processing time due to storing intermediate results

Since intermediate results are always saved to disk, extra time gets added at every stage just for storing and retrieving this data, increasing the overall time taken to complete a job.

6. Less flexible for modern applications

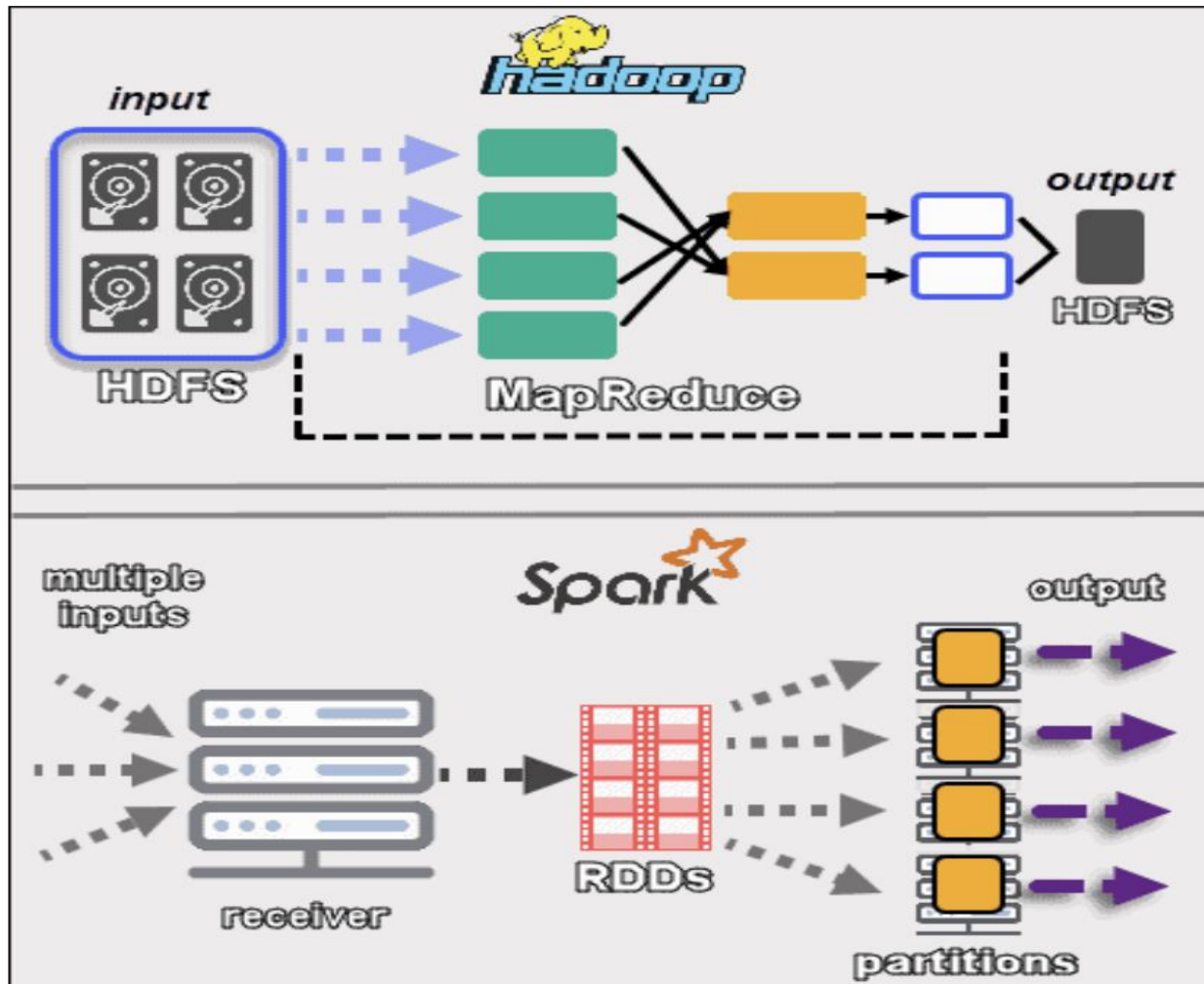
MapReduce isn't very flexible for the kind of work modern applications need, like real-time analytics, machine learning, and interactive queries, since it wasn't built with these use cases in mind.

Why Spark is preferred:

Instead of writing to disk at every step, Spark keeps the data in memory, so it can be reused instantly across different operations without reloading it each time. This is the main reason Spark performs so much faster, especially for jobs that require multiple iterations. Spark is also easier

to code in, and it can handle batch processing, real-time streaming, SQL-based queries, and machine learning all within a single framework, something MapReduce cannot do on its own.

So in short, MapReduce's dependency on disk and its rigid structure are what hold it back, while Spark's ability to process data in memory and handle multiple types of workloads is what makes it the more practical and preferred choice for big data processing today.



Looking at this diagram, it shows the difference between how MapReduce and Spark process data. MapReduce reads input from **HDFS (Hadoop Distributed File System)**, sends it through Map and Reduce tasks, and then writes the output back to disk at every stage, which is why it ends up being slower. Spark works differently — it takes in data through a receiver and converts it into **RDDs (Resilient Distributed Datasets)**, which are then processed in memory using partitions. Since Spark avoids going back to the disk at every step, it performs much faster, especially when the same data needs to be processed more than once.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

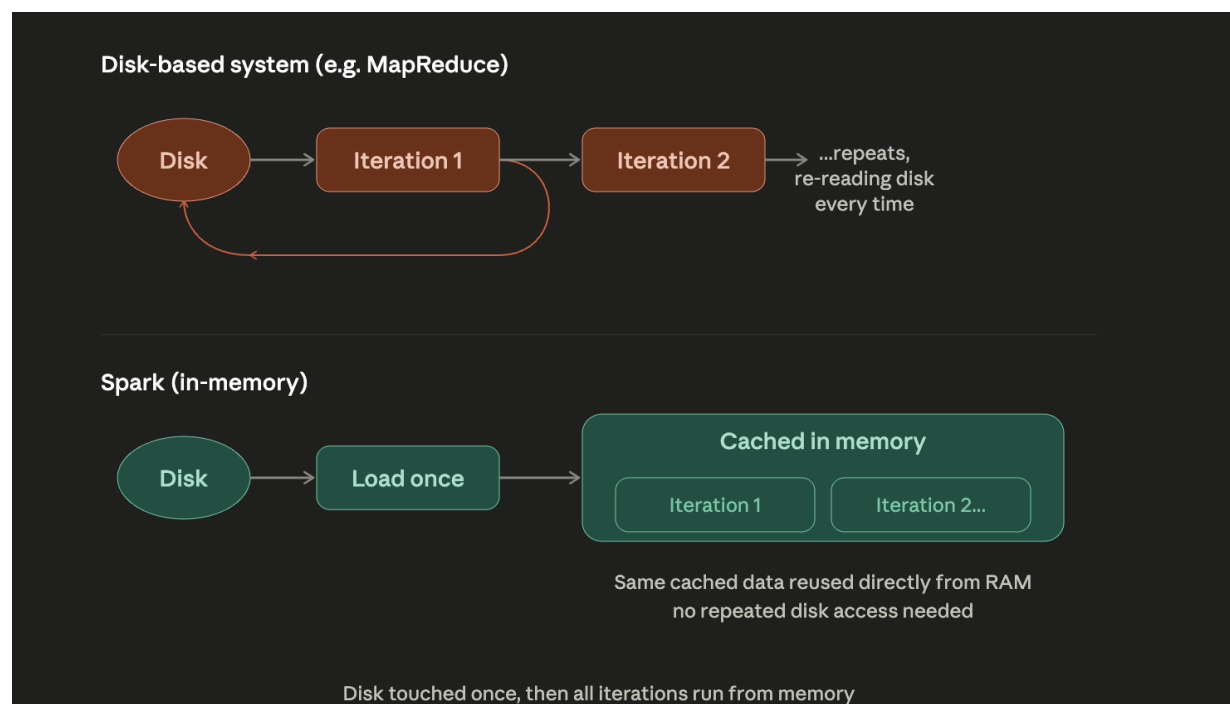
Solution:

Machine learning algorithms are iterative, meaning they go through the same data multiple times to slowly improve the model with each pass. This repeated processing is where the difference between Spark and traditional disk-based systems really shows up.

In systems like Hadoop MapReduce, the intermediate data generated during each step has to be written to disk and then read back again before the next step can begin. Since disk access is much slower compared to memory, this constant writing and reading ends up taking a lot of extra time, especially when the same process has to repeat again and again.

Spark handles this differently. Instead of writing intermediate data to disk after every step, it keeps that data stored in RAM. So when the next iteration needs the same data, Spark can just pull it directly from memory instead of going back to the disk to fetch it.

Because of this, Spark avoids most of the repeated disk I/O that slows down traditional systems, which is exactly why it performs so much faster for machine learning algorithms that need to process the same data multiple times.



This diagram shows that a disk-based system reads data from disk again for every iteration, adding delay each time. Spark, on the other hand, loads the data into memory only once and

reuses it directly from RAM for all iterations, which is why it performs much faster for repeated ML computations.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Solution:

```
df_clean = df.dropDuplicates(["Customer ID", "Order Date"])
df_clean.show()
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal
1300	CA-2015-121391	10/4/2015	10/7/2015	First Class	AA-10315	Alex Avila	Consumer	United States	San Francisco	California	94109
5199	CA-2016-103982	3/3/2016	3/8/2016	Standard Class	AA-10315	Alex Avila	Consumer	United States	Round Rock	Texas	78664
2230	CA-2014-128055	3/31/2014	4/5/2014	Standard Class	AA-10315	Alex Avila	Consumer	United States	San Francisco	California	94122
1160	CA-2017-147039	6/29/2017	7/4/2017	Standard Class	AA-10315	Alex Avila	Consumer	United States	Minneapolis	Minnesota	55407
7469	CA-2014-138100	9/15/2014	9/20/2014	Standard Class	AA-10315	Alex Avila	Consumer	United States	New York City	New York	10011
3008	CA-2014-130729	10/24/2014	10/29/2014	Standard Class	AA-10375	Allen Arnold	Consumer	United States	Rancho Cucamonga	California	91730
6466	CA-2015-114503	11/13/2015	11/17/2015	Standard Class	AA-10375	Allen Arnold	Consumer	United States	Lebanon	Tennessee	37087
2264	CA-2016-131065	11/14/2016	11/16/2016	Second Class	AA-10375	Allen Arnold	Consumer	United States	Atlanta	Georgia	30318
6748	CA-2017-100230	12/11/2017	12/15/2017	Standard Class	AA-10375	Allen Arnold	Consumer	United States	New York City	New York	10035
808	CA-2015-140921	2/3/2015	2/5/2015	First Class	AA-10375	Allen Arnold	Consumer	United States	Omaha	Nebraska	68104
1173	CA-2014-158064	4/21/2014	4/25/2014	Standard Class	AA-10375	Allen Arnold	Consumer	United States	Los Angeles	California	90008
1979	CA-2015-109939	5/8/2015	5/12/2015	Standard Class	AA-10375	Allen Arnold	Consumer	United States	Salem	Oregon	97301
536	CA-2016-126613	7/10/2016	7/16/2016	Standard Class	AA-10375	Allen Arnold	Consumer	United States	Mesa	Arizona	85204
9539	US-2017-169488	9/7/2017	9/9/2017	First Class	AA-10375	Allen Arnold	Consumer	United States	Providence	Rhode Island	02802
13	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord	North Carolina	28027
1460	CA-2014-155271	5/4/2014	5/4/2014	Same Day	AA-10480	Andrew Allen	Consumer	United States	Middletown	Connecticut	06457
3108	CA-2016-121671	7/17/2016	7/22/2016	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Springfield	Missouri	65807
7703	CA-2016-114601	8/26/2016	9/2/2016	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Detroit	Michigan	48234
8008	CA-2015-110863	11/17/2015	11/24/2015	Standard Class	AA-10645	Anna Andreadi	Consumer	United States	Oklahoma City	Oklahoma	73120
7490	CA-2017-157196	11/5/2017	11/9/2017	Standard Class	AA-10645	Anna Andreadi	Consumer	United States	San Diego	California	92105

only showing top 20 rows

```
df_clean = df.dropDuplicates(["Customer ID", "Order Date"])
df_clean.show()
```

City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
San Francisco	California	94109	West	OFF-ST-10001590	Office Supplies	Storage	Tenex Personal Pr...	26.96	2	0.0	7.0096
Round Rock	Texas	78664	Central	OFF-SU-10000151	Office Supplies	Supplies	High Speed Automa...	3930.072	3	0.2	-786.0144
San Francisco	California	94122	West	OFF-BI-10004390	Office Supplies	Binders	GBC DocuBind 200 ...	673.568	2	0.2	252.588
Minneapolis	Minnesota	55407	Central	OFF-AP-10000576	Office Supplies	Appliances	Belkin 325VA UPS ...	362.94	3	0.0	90.735
New York City	New York	10011	East	OFF-PA-10000349	Office Supplies	Paper	Easy-staple paper	14.94	3	0.0	7.0218
Rancho Cucamonga	California	91730	West	OFF-BI-10002706	Office Supplies	Binders	Avery Premier Hea...	34.272	3	0.2	11.1384
Lebanon	Tennessee	37087	South	OFF-ST-10003572	Office Supplies	Storage	Portfile Personal...	84.96	6	0.2	6.372
Atlanta	Georgia	30318	South	TEC-AC-10004145	Technology	Accessories	Logitech diNovo E...	499.98	2	0.0	114.9954
New York City	New York	10035	East	OFF-BI-10002498	Office Supplies	Binders	Clear Mylar Reinf...	14.952	1	0.2	5.0463
Omaha	Nebraska	68104	Central	FUR-FU-10003347	Furniture	Furnishings	Coloredge Poster ...	28.4	2	0.0	11.076
Los Angeles	California	90008	West	OFF-BI-10002976	Office Supplies	Binders	ACCOHIDE Binder b...	16.52	5	0.2	5.5755
Salem	Oregon	97301	West	OFF-AR-10000127	Office Supplies	Art	Newell 321	5.248	2	0.2	0.5904
Mesa	Arizona	85204	West	OFF-ST-10001325	Office Supplies	Storage	Sterilite Officew...	16.768	2	0.2	1.4672
Providence	Rhode Island	02802	East	OFF-PA-10002659	Office Supplies	Paper	Avoid Verbal Orde...	16.9	5	0.0	7.774
Concord	North Carolina	28027	South	OFF-PA-10002365	Office Supplies	Paper	Xerox 1967	15.552	3	0.2	5.4432
Middletown	Connecticut	06457	East	FUR-FU-10001473	Furniture	Furnishings	DAX Wood Document...	27.46	2	0.0	9.8856
Springfield	Missouri	65807	Central	OFF-PA-10001471	Office Supplies	Paper	Strathmore Photo ...	21.93	3	0.0	10.0878
Detroit	Michigan	48234	Central	OFF-PA-10000605	Office Supplies	Paper	Xerox 1950	11.56	2	0.0	5.6644
Oklahoma City	Oklahoma	73120	Central	OFF-ST-10002756	Office Supplies	Storage	Tennsco Stur-D-St...	541.24	4	0.0	5.4124
San Diego	California	92105	West	OFF-PA-10003172	Office Supplies	Paper	Xerox 1996	12.96	2	0.0	6.2208

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Solution:

```

▶ from pyspark.sql import functions as F

result = (df.filter(F.col("Region") == "West")
          .groupBy("Category")
          .agg(F.avg("Sales").alias("avg_sale_amount")))
result.show()

```

```

... +-----+-----+
|      Category| avg_sale_amount|
+-----+-----+
|Office Supplies| 116.422376910912|
|      Furniture| 357.3023246110322|
|      Technology|420.68753255425725|
+-----+-----+

```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Solution:

```

df.na.fill({"Ship Mode": "Unknown"}).select("Ship Mode").distinct().show()

```

```

+-----+
| Ship Mode|
+-----+
| First Class|
| Same Day|
| Second Class|
| Standard Class|
+-----+

```

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Solution:

```
city_counts = df.groupBy("City").count().filter(F.col("count") > 100)
city_counts.show()
```

...

City	count
Springfield	163
Dallas	157
Philadelphia	537
Los Angeles	747
San Francisco	510
San Diego	170
Detroit	115
Columbus	222
Chicago	314
Seattle	428
New York City	915
Houston	377
Jacksonville	125

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Solution:

- Spark DataFrames are immutable, meaning once we create one, we can't actually change it directly.
- So whenever we do something like dropping a column or renaming one, Spark doesn't touch the original DataFrame at all — it just creates a brand new one with those changes applied.
- For example, if we run `df.drop("Postal Code")`, the original `df` stays exactly the same as before; we'd need to save the result in a new variable if we actually want to use the cleaned version.
- Because of this, cleaning data in Spark usually happens step by step, where we apply one transformation at a time and save it into a new (or the same) variable, instead of just editing the DataFrame in place like we might with a normal list or dictionary in Python.
- This is actually pretty useful too, since it means our original data stays untouched no matter how many cleaning steps we try, so there's less risk of accidentally messing up the source data.
- Overall, immutability makes the whole process feel a bit safer and more predictable, since every change just gives us a new version of the DataFrame instead of quietly overwriting the original one.

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Solution:

```
q8 = df.filter((F.col("Sales").between(100, 500)) & (F.col("Segment") == "Consumer"))
q8.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky
10	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California
14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washington
20	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	California
30	US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	Tracy Blumstein	Consumer	United States	Philadelphia	Pennsylvania

only showing top 5 rows

Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
42420	South	FUR-B0-10001798	Furniture	Bookcases	Bush Somerset Col...	261.96	2	0.0	41.9136
90032	West	OFF-AP-10002892	Office Supplies	Appliances	Belkin F5C206VTEL...	114.9	5	0.0	34.47
98103	West	OFF-BI-10003656	Office Supplies	Binders	Fellowes PB200 Pl...	407.976	3	0.2	132.5922
94109	West	TEC-PH-10001949	Technology	Phones	Cisco SPA 501G IP...	213.48	3	0.2	16.011
19140	East	FUR-FU-10004848	Furniture	Furnishings	Howard Miller 13-...	124.2	3	0.2	15.525

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Solution:

- Missing values can quietly affect the accuracy of aggregation results if left unhandled.
- For example, in the Superstore dataset, if the Sales column had null values, calculating the total or average sales without addressing them first could give results that look fine but are actually based on incomplete data.
- Aggregation functions like sum() and avg() can end up ignoring these missing values silently, without any warning that some data was skipped.
- That's why it's better to check for nulls first and handle them, either by removing those rows using .na.drop() or filling them with a suitable value using .na.fill().
- Doing this before running sum() or avg() makes sure the final result actually reflects the real data, instead of unknowingly leaving out missing values.

Q10: Write the code to **revise** a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

Solution:

```
df_updated = (df.withColumn("event_time", F.to_timestamp(F.col("Order Date"), "M/d/yyyy"))
                .drop("Order Date"))
df_updated.select("event_time").show(5)
```

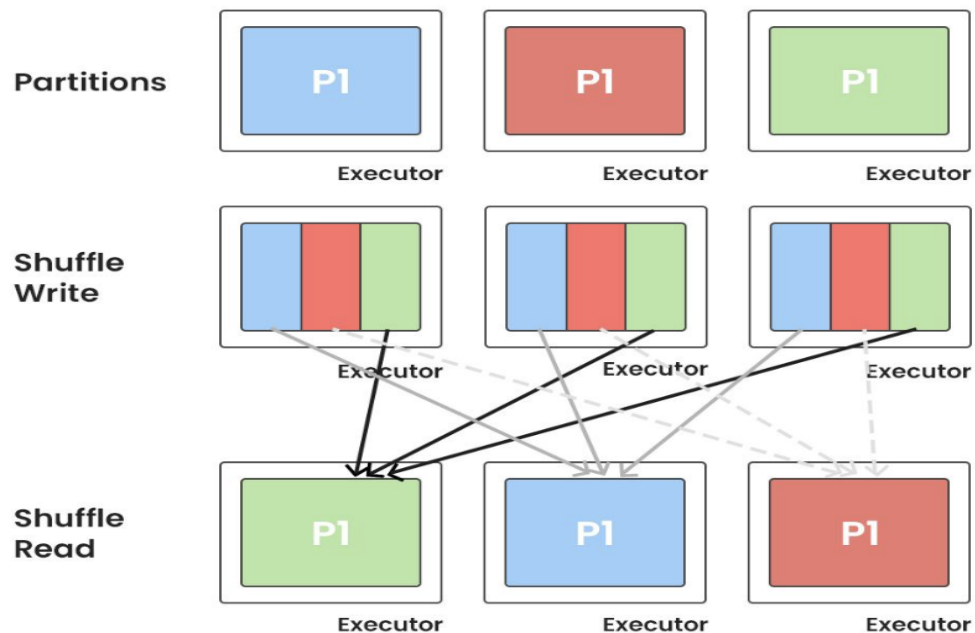
```
+-----+
|      event_time|
+-----+
|2016-11-08 00:00:00|
|2016-11-08 00:00:00|
|2016-06-12 00:00:00|
|2015-10-11 00:00:00|
|2015-10-11 00:00:00|
+-----+
only showing top 5 rows
```

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Solution:

- When we run something like a `groupBy`, Spark needs all the rows with the same key to end up together so they can actually be aggregated properly.
- The thing is, matching keys are usually scattered across different partitions in the cluster, so Spark has to move data around — reading it from one place and sending it over to another — and this whole process is what we call a shuffle.
- It involves writing intermediate data to disk and sending it across the network to the right partition, which is why it ends up being one of the more expensive things Spark does.
- It's called a wide transformation because each output partition can end up depending on data coming from many different input partitions, unlike something like `filter` or `map`, where each output partition only needs data from just one input partition.
- Since shuffles are so costly, they're usually the main reason a Spark job slows down, so reducing unnecessary shuffling (like filtering the data before doing a `groupBy`) is one of the best ways to make a job run faster.

How Spark Architecture Shuffle Works ?



This diagram shows how shuffle works step by step. Each executor starts with its own partition of data. During "shuffle write," it splits that data into smaller parts based on key (shown by the different colors). Then during "shuffle read," each executor collects all the matching-key parts from the other executors, so all the data for one key ends up together. This is why `groupBy` is called a wide transformation — the final result depends on data from all the partitions, not just one.

Q12: Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

Solution:

```
[22] ✓ 0s q12 = df.filter(F.col("Customer ID").isNotNull() & (F.trim(F.col("Customer Name")) != ""))
q12.count()
... 9994
```

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Solution:

- The .agg() function is pretty handy because it lets us calculate several statistics together in just one go, instead of writing a separate line for each one.
- All we need to do is pass multiple aggregate expressions inside .agg(), and Spark works out all of them at the same time.
- This ends up being more efficient too, since Spark only has to scan through the data once, instead of scanning it three separate times for min, max, and mean individually.

```
[23] df.agg(F.min("Sales").alias("min_price"),
        F.max("Sales").alias("max_price"),
        F.mean("Sales").alias("mean_price")).show()
```

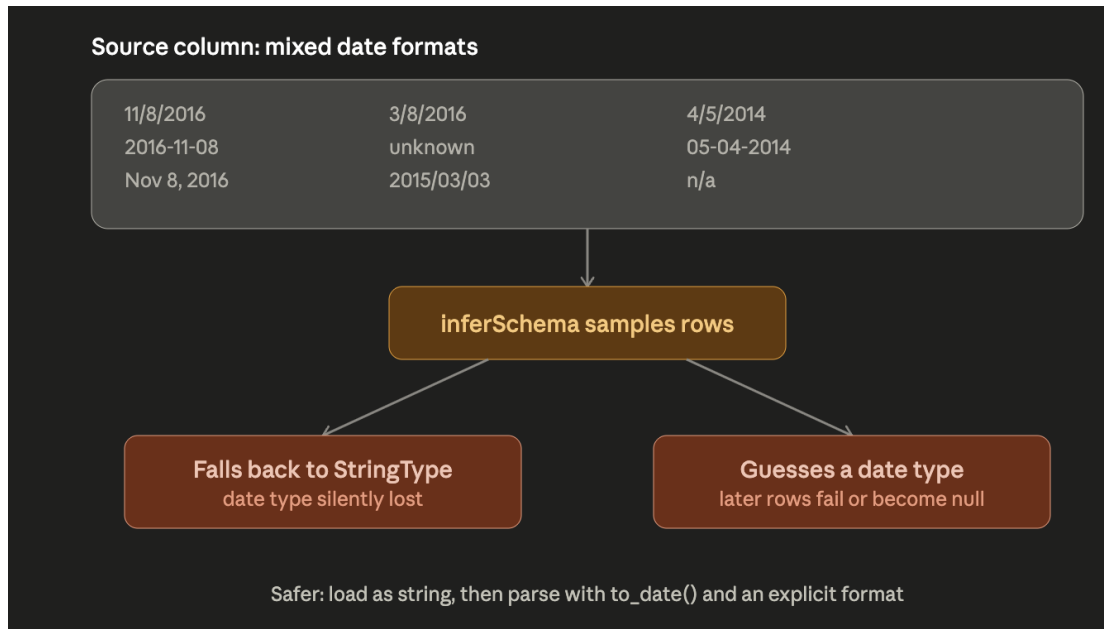
min_price	max_price	mean_price
0.444	22638.48	229.85800083049725

Q14: In the context of **cleaning a dataset**, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Solution:

- inferSchema=true basically makes Spark look at a sample of the data and try to guess what type each column should be.
- The problem is, if a date column has mixed formats, like some rows as "MM/DD/YYYY" and others as "YYYY-MM-DD" or even plain text, Spark can get confused about what the actual type should be.
- A lot of the time, it just ends up treating the whole column as plain text instead, so we lose the date/timestamp type we actually wanted, without even realizing it happened.
- And even if it does manage to guess a date type based on whatever sample it looked at, it can still break or return nulls later on when it comes across rows that don't match that same format.
- On top of that, it slows things down a bit too, since inferSchema has to scan through the data first before it can decide on the types.

- So it's usually a safer bet to just load these kinds of columns as strings first, and then convert them ourselves using something like `to_date()` with the exact format specified, so if there's a parsing issue, we actually see it happen instead of it quietly getting hidden.



Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by `store_id` to calculate total revenue.

Solution:

[24]

✓ 2s

```
▶ pipeline_result = (df.dropDuplicates()  
                      .na.fill({"Sales": 0})  
                      .groupBy("Region")  
                      .agg(F.sum("Sales").alias("total_revenue"))  
                      pipeline_result.show())
```

▼

```
... +-----+-----+  
| Region| total_revenue|  
+-----+-----+  
|   South|      391721.905|  
|Central|501239.89079999994|  
|   East| 678781.2400000008|  
|   West| 725457.8245000013|  
+-----+-----+
```